

AI Integration in PHP Applications

Building your AI-powered web application with clean architecture, prompt engineering, and the service pattern.

READING OVERVIEW

This is the capstone of your PHP and Laravel journey. You will integrate a third-party AI service into your blog application using the same patterns professional teams use: a dedicated service class, careful prompt engineering, honest error handling, and testable code. Along the way you will see how the framework has begun to productize this exact pattern, and where to go next.

LEARNING OBJECTIVES

- Write effective prompts using role prompting and clear, specific instructions.
- Build an AI draft generator across a Blade view, a route, a controller, and a service class.
- Store an API key and model name safely in `.env` and read them through config.
- Handle failure deliberately with `Throwable`, logging, caching, and fallbacks.

- Test AI code by faking the HTTP layer, and know when to reach for `laravel/ai`.

01 Prompt Engineering: The Art of AI Communication

Imagine giving directions to someone who is brilliant and capable but takes everything you say completely literally. That person can help you accomplish amazing things, but only if you communicate clearly and precisely. That is what working with an AI system is like, and it is why a skill called prompt engineering has become one of the more valuable abilities in modern software work.

Prompt engineering is the practice of crafting precise, effective instructions for AI systems to achieve a desired outcome. In its first wave it was sometimes advertised as a standalone job with headline salaries. The market has since matured. Prompt engineering is now most often an expected skill embedded within broader roles such as AI engineer, software engineer, product manager, and technical writer, rather than a separate title. The practical takeaway for you is that being able to direct an AI system well raises your value across many roles, which is exactly what this module helps you build.

Career Connection

You do not need the title “Prompt Engineer” on a job posting for this skill to pay off. Listings for AI engineer, backend engineer, and product roles increasingly list “experience integrating LLMs” as a requirement. The feature you build in this module is concrete evidence you can point to in an interview.

The principle underneath all effective prompting is one you already know from programming: garbage in, garbage out. Just as a function produces good results only when given good inputs, an AI system generates valuable output only when given a

well-crafted prompt. The difference between a vague request and a precise one is the difference between generic filler and content you can actually use.

Role Prompting: Setting the Stage

One of the most reliable techniques is role prompting, where you establish a specific persona or area of expertise for the AI to adopt. Think of it as casting the model in a part. When you open a prompt with “You are a witty tech blogger who makes complex programming concepts accessible to beginners,” you are not just adding information. You are changing how the model approaches the whole task.

The effect is easy to see in practice. Support teams use role prompting so their chatbots respond with the right tone and company knowledge. Marketing teams use it to keep a consistent brand voice. Documentation teams use it so explanations match their audience. For your blog application, “You are a helpful programming tutor who explains ideas through relatable analogies” will produce very different content than “You are a formal business analyst who provides data-driven insights.” Learning how these personas shape output is a skill that carries across every kind of project.

Crafting Clear Instructions

Beyond setting a role, effective prompting means giving specific, actionable instructions about length, tone, format, and scope. Rather than asking the model to “write something about databases,” a well-engineered prompt specifies: “Write a 300-word explanation of the difference between SQL and NoSQL databases, in a conversational tone for someone who just learned basic HTML and CSS. Include one concrete example of when you would choose each, and use clear subheadings.”

More advanced practitioners use few-shot prompting, where they supply examples of the output they want, and chain-of-thought prompting, where they ask the model to show its reasoning. These help most when an application needs consistent, high-quality output at scale. The same strategies show up well beyond content generation: code review, documentation, ticket triage, and analysis all lean on the same skill you are practicing here.

Building Your AI-Powered Feature: Concept to Code

Now that you understand the principles of AI communication, let us build a feature that puts them to work: an AI-powered blog post draft generator. A user enters a title and receives a well-structured draft to start from.

This feature is a clean demonstration of separation of concerns, the principle you have been building toward all course. Our system has four layers, each with one job. The user interface layer handles presentation and input through Blade templates and forms. The controller layer manages the request and coordinates the process. The service layer holds all the logic for talking to the external AI API. The external API layer is the OpenAI service that actually generates the content.

This layered approach mirrors what large teams do in production. When you look at how companies integrate AI into merchant tools or messaging platforms, you see the same separation between interface, business logic, and external service. This is not just good for learning. It is the same shape you will use professionally.

The User Interface Layer

We start by extending the existing create post view to include AI generation. The key idea is progressive enhancement: the feature should feel like a natural part of the existing workflow, not a bolted-on system.

```
{{-- resources/views/posts/create.blade.php --}}
@extends('layouts.app')

@section('content')
<div class="container mx-auto px-4 py-6">
    <h1 class="text-3xl font-bold mb-6">Create New Post</h1>

    {{-- AI Draft Generation Section --}}
    <div class="bg-blue-50 border border-blue-200 rounded-lg p-6 mb-6">
        <h2 class="text-xl font-semibold mb-4 text-blue-800">AI Draft Generator</h2>
        <p class="text-blue-700 mb-4">Get a head start with an AI-generated draft by
```

```

<form action="{{ route('posts.generate-draft') }}" method="POST" class="mb-4">
    @csrf
    <div class="flex gap-3">
        <input type="text"
            name="title"
            placeholder="Enter your post title..."
            class="flex-1 px-3 py-2 border border-blue-300 rounded-md focus:outline-none"
            value="{{ old('title') }}"
            required>
        <button type="submit"
            class="bg-blue-600 hover:bg-blue-700 text-white px-6 py-2 rounded-md">
            Generate Draft
        </button>
    </div>
</form>

{{-- Display any generation errors --}}
@if ($errors->has('ai_generation'))
    <div class="text-red-600 text-sm">{{ $errors->first('ai_generation') }}</div>
@endif
</div>

{{-- Main Post Creation Form --}}
<form action="{{ route('posts.store') }}" method="POST">
    @csrf
    <div class="mb-4">
        <label for="title" class="block text-sm font-medium text-gray-700 mb-2">Title</label>
        <input type="text"
            id="title"
            name="title"
            class="w-full px-3 py-2 border border-gray-300 rounded-md focus:outline-none"
            value="{{ old('title', $generatedTitle ?? '') }}"
            required>
        @error('title')
            <div class="text-red-600 text-sm mt-1">{{ $message }}</div>
        @enderror
    </div>

    <div class="mb-4">
        <label for="body" class="block text-sm font-medium text-gray-700 mb-2">Body</label>
        <textarea id="body"
            name="body"
            rows="12"
            class="w-full px-3 py-2 border border-gray-300 rounded-md focus:outline-none">

```

```

        required>{{ old('body', $generatedContent ?? '') }}</textarea>
        @error('body')
            <div class="text-red-600 text-sm mt-1">{{ $message }}</div>
        @enderror
    </div>

    <div class="flex justify-end">
        <button type="submit"
            class="bg-green-600 hover:bg-green-700 text-white px-6 py-3 rounded"
            Create Post
        </button>
    </div>
</form>
</div>
@endsection

```

A few user-experience ideas are working here. The AI section is visually distinct but clearly part of the main form. We use progressive enhancement: a separate form generates the draft and then populates the main creation form. The `old()` helper preserves user input even after a validation error or an AI generation round-trip. When a user generates a draft, we send them back to this same view with the generated content pre-filled, so the AI feature feels like a seamless part of writing a post.

The Route and Controller Logic

Next we define the route that handles generation requests and the controller method that coordinates the process.

```

// routes/web.php
Route::middleware('auth')->group(function () {
    // Existing post routes
    Route::resource('posts', PostController::class);

    // New AI generation route
    Route::post('/posts/generate-draft', [PostController::class, 'generateDraft'])
        ->name('posts.generate-draft');
});

```

The route follows RESTful conventions while creating a specific endpoint for the AI feature. Placing it inside the auth middleware group means only logged-in users can reach this potentially expensive feature. Now the controller method that handles generation:

```
// app/Http/Controllers/PostController.php
<?php

namespace App\Http\Controllers;

use App\Models\Post;
use App\Services\AiContentService;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;

class PostController extends Controller
{
    /**
     * Generate an AI draft for a blog post based on the provided title.
     */
    public function generateDraft(Request $request)
    {
        // Validate the incoming request
        $request->validate([
            'title' => 'required|string|max:255|min:3',
        ]);

        try {
            $title = $request->input('title');

            // Instantiate our service. In a larger app you would inject this
            // through the constructor rather than newing it up here.
            $aiContentService = new AiContentService();

            // The service handles all the API communication details.
            $generatedContent = $aiContentService->generateBlogPostDraft($title);

            Log::info('AI blog post draft generated', [
                'user_id' => Auth::id(),
                'title' => $title,
                'content_length' => strlen($generatedContent),
            ]);
        } catch (\Exception $e) {
            Log::error('AI blog post draft generation failed: ' . $e->getMessage());
        }
    }
}
```

```

    });

    // Return to the create view with the generated content.
    return view('posts.create', [
        'generatedTitle'    => $title,
        'generatedContent' => $generatedContent,
        'successMessage'    => 'AI draft generated. You can edit it below bef
    ]);

} catch (\Throwable $e) {
    // Handle API errors, network issues, or service failures.
    Log::error('AI generation failed', [
        'user_id' => Auth::id(),
        'title'   => $request->input('title'),
        'error'   => $e->getMessage(),
    ]);

    return back()
        ->withInput()
        ->withErrors(['ai_generation' => 'Unable to generate a draft right r
    }
}
}

```

This method shows several professional habits at once. The error handling means an API failure does not break the user experience. The logging gives you visibility into what happened for monitoring and debugging. The validation protects an expensive API call from bad input. And the clean split between controller and service keeps the code maintainable and testable.

⚙️ Professional Practice

Notice the catch block uses `\Throwable` rather than `\Exception`. `Exception` only catches deliberate exceptions your code raises. `Throwable` also catches `Error`, the deeper problems PHP itself raises, such as a type mismatch or calling a method on null. The Laravel convention is to catch broadly with `Throwable` and throw specifically with a named exception class.

The Service Layer: Clean Architecture in Action

The service layer holds all the logic for talking to the OpenAI API. Isolating it in one dedicated class is what makes the code maintainable, testable, and easy to change later if you switch AI providers.

```
// app/Services/AiContentService.php
<?php

namespace App\Services;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

/**
 * OpenAI Service Class
 *
 * Handles all communication with the OpenAI API. Isolating this in a
 * dedicated class follows the Single Responsibility Principle and makes
 * the code more maintainable, testable, and flexible.
 */
class AiContentService
{
    private string $apiKey;
    private string $baseUrl;
    private string $model;

    public function __construct()
    {
        // Load configuration from environment variables so secrets stay
        // out of the codebase.
        $this->apiKey = config('services.openai.key');
        $this->baseUrl = config('services.openai.url', 'https://api.openai.com/v1');
        $this->model = config('services.openai.model', 'gpt-4o-mini');

        // Fail fast if the key is missing.
        if (empty($this->apiKey)) {
            throw new \Exception('OpenAI API key not configured');
        }
    }

    /**
```

```

    * Generate a blog post draft based on a given title.
    */
    public function generateBlogPostDraft(string $title): string
    {
        $prompt    = $this->buildBlogPostPrompt($title);
        $response = $this->makeOpenAIRequest($prompt);

        return $this->extractContentFromResponse($response);
    }

```

```

/**
 * Build a well-engineered prompt for blog post generation.
 */
private function buildBlogPostPrompt(string $title): string
{

```

return "You are a skilled tech blogger who writes engaging, informative content for developers and technology enthusiasts. Your writing style is conversational yet professional and you excel at making complex topics accessible to a broad audience.

Task: Write a complete blog post draft based on the title: "{\$title}"

Requirements:

- Length: 400-600 words
- Include an engaging introduction that hooks the reader
- Provide 2-3 main sections with clear subheadings
- Include practical examples or code snippets when relevant
- End with a conclusion that summarizes key points
- Use a tone that is informative but approachable

Format the response as clean markdown with proper headings and structure."

```

/**
 * Make the actual HTTP request to the OpenAI API.
 */
private function makeOpenAIRequest(string $prompt): array
{
    try {
        $response = Http::withHeaders([
            'Authorization' => 'Bearer ' . $this->apiKey,
            'Content-Type'  => 'application/json',
        ])
            ->timeout(30) // 30-second timeout for API requests
            ->post($this->baseUrl . '/chat/completions', [

```

```

        'model'      => $this->model,
        'messages' => [
            ['role' => 'user', 'content' => $prompt],
        ],
        'max_tokens' => 1000,
        'temperature' => 0.7, // Balanced creativity and consistency
    ]);

    if (!$response->successful()) {
        throw new \Exception('OpenAI API request failed: ' . $response->body);
    }

    return $response->json();

} catch (\Throwable $e) {
    Log::error('OpenAI API communication failed', [
        'error'      => $e->getMessage(),
        'prompt_length' => strlen($prompt),
    ]);

    throw new \Exception('Failed to communicate with AI service: ' . $e->getMessage());
}

}

/**
 * Extract the generated content from the OpenAI API response.
 */
private function extractContentFromResponse(array $response): string
{
    if (!isset($response['choices'][0]['message']['content'])) {
        throw new \Exception('Unexpected response format from OpenAI API');
    }

    $content = trim($response['choices'][0]['message']['content']);

    if (empty($content)) {
        throw new \Exception('No content generated by AI service');
    }

    return $content;
}
}

```

This class demonstrates several ideas that matter in professional work. The constructor validates configuration and fails fast if it is missing. The private methods break a complex interaction into small, testable pieces. The error handling and logging give you visibility. Most importantly, the service encapsulates all OpenAI-specific logic, so switching providers or changing the integration does not ripple through the rest of the application.

Before the service can run, add the OpenAI configuration to your application:

```
// config/services.php
return [
    // ... existing service configurations

    'openai' => [
        'key'    => env('OPENAI_API_KEY'),
        'url'    => env('OPENAI_API_URL', 'https://api.openai.com/v1'),
        'model' => env('OPENAI_MODEL', 'gpt-4o-mini'),
    ],
];
```

And add your key and model to the .env file, which is never committed to version control:

```
OPENAI_API_KEY=your_api_key_here
OPENAI_MODEL=gpt-4o-mini
```

⚠ Watch Out

We use `gpt-4o-mini`, the current low-cost, fast model well suited to draft generation. The model that earlier versions of this course used, `gpt-3.5-turbo`, is being retired from the OpenAI API on October 23, 2026, so do not build against it. Because the model name lives in `.env`, you can swap it in one place as the lineup changes, without touching your service code.

⚡ Modern PHP

Everything above builds the AI integration by hand so you can see exactly what an SDK abstracts: the HTTP call, the headers, the JSON shape, and the error handling. That is the point of the exercise. In production on the Laravel 13 baseline, though, you would most often reach for the framework's first-party AI package, `laravel/ai`, which productizes this exact service pattern. The community package Prism does the same and lets you switch providers by changing one line.

The same request through `laravel/ai` looks roughly like this:

```
// The modern idiom: a first-party Agent class.
// composer require laravel/ai

use Laravel\Ai\Attributes\Model;
use Laravel\Ai\Attributes\Provider;
use Laravel\Ai\Contracts\Agent;
use Laravel\Ai\Concerns\Promptable;
use Laravel\Ai\Enums\Lab;

#[Provider(Lab::OpenAI)]
#[Model('gpt-4o-mini')]
class BlogDraftAgent implements Agent
{
    use Promptable;

    public function instructions(): string
    {
        return 'You are a skilled tech blogger who writes engaging, informativ';
    }
}
```

You will build the hand-rolled service first because it teaches the concepts. Rebuilding it on top of `laravel/ai` or Prism is the recommended stretch goal, and it is the direction real Laravel jobs now expect.

Handling Rate Limits and Quotas

One of the most important things to manage with any external AI service is rate limiting. Providers set these limits to keep the system fair and stable, but they will hurt your user experience if you do not handle them. Here is the service enhanced to track and respect a simple rate limit.

```
// Enhanced AiContentService with rate limiting
public function generateBlogPostDraft(string $title): string
{
    // Check if we are approaching the rate limit.
    if ($this->isNearRateLimit()) {
        throw new \Exception('API rate limit approaching. Please try again in a few
    }

    $this->recordApiCall();

    $prompt    = $this->buildBlogPostPrompt($title);
    $response = $this->makeOpenAIRequest($prompt);

    return $this->extractContentFromResponse($response);
}

private function isNearRateLimit(): bool
{
    // A simplified example. Real code would check recent calls in cache or database
    $recentCalls = cache()->get('openai_calls_' . now()->format('Y-m-d-H'), 0);
    return $recentCalls ≥ 50; // Assuming a 60-calls-per-hour limit
}

private function recordApiCall(): void
{
    $key = 'openai_calls_' . now()->format('Y-m-d-H');
    cache()->increment($key, 1, 3600); // Store for one hour
}
```

Caching for Cost Optimization

AI API calls cost time and money. Intelligent caching can improve performance and cut costs at the same time. If two users request a draft for the same title, there is no reason to pay for the work twice.

```
public function generateBlogPostDraft(string $title): string
{
    // Build a cache key from the normalized title.
    $cacheKey = 'ai_draft_' . md5(strtolower(trim($title)));

    // Return a cached result if we have one.
    $cachedContent = cache()→get($cacheKey);
    if ($cachedContent) {
        Log::info('Retrieved AI content from cache', ['title' ⇒ $title]);
        return $cachedContent;
    }

    // Otherwise generate fresh content.
    $prompt    = $this→buildBlogPostPrompt($title);
    $response  = $this→makeOpenAIRequest($prompt);
    $content   = $this→extractContentFromResponse($response);

    // Cache the result for 24 hours.
    cache()→put($cacheKey, $content, 86400);

    return $content;
}
```

Quick Tip

Caching AI responses is one of the highest-leverage cost controls you can add. Providers also offer prompt caching that discounts repeated input tokens automatically, so structuring your prompts with the stable parts first (the system instructions) and the variable parts last (the user's title) can lower your bill without any extra code.

Building Multiple AI Features

As your application grows you will want more than one AI feature. A small dispatch method lets the service support several content types while keeping each prompt

focused.

```
/**
 * Generate different types of AI content based on the requested type.
 */
public function generateContent(string $input, string $type = 'blog_post'): string
{
    return match ($type) {
        'blog_post'      => $this->generateBlogPostDraft($input),
        'social_media'   => $this->generateSocialMediaPost($input),
        'meta_description' => $this->generateMetaDescription($input),
        default          => throw new \InvalidArgumentException("Unsupported content type")
    };
}

private function generateSocialMediaPost(string $topic): string
{
    $prompt = "You are a social media expert who creates engaging posts for technology products.

    Task: Create a social media post about: \"{ $topic }\"

    Requirements:
    - Length: 50-100 words
    - Include 2-3 relevant hashtags
    - Use an engaging, conversational tone
    - Include a call to action";

    $response = $this->makeOpenAIRequest($prompt);
    return $this->extractContentFromResponse($response);
}

private function generateMetaDescription(string $title): string
{
    $prompt = "You are an SEO expert who writes compelling meta descriptions.

    Task: Write a meta description for a blog post titled: \"{ $title }\"

    Requirements:
    - 150-160 characters
    - Include relevant keywords
    - Be compelling and clickable
    - Accurately describe the content";
```



```
$response = $this->makeOpenAIRequest($prompt);  
return $this->extractContentFromResponse($response);  
}
```

⚡ Modern PHP

The `match` expression above is the modern replacement for a long `switch` statement. It is stricter (it compares with `===`), it returns a value directly, and it throws automatically if nothing matches. You will see it everywhere in current PHP codebases.

04 Industry Context: AI in Modern Web Applications

The Current State of AI in Web Development

AI in web applications has moved from experimental feature to expected capability across nearly every industry. In recent years, user expectations have shifted fundamentally. Applications that do not use AI capabilities are increasingly seen as dated or incomplete.

Major platforms led this shift. Coding assistants changed how developers write code. AI-generated product descriptions help online stores create content at scale. Conversation summaries help teams catch up quickly. Content tools assist with drafting, analysis, and workflow automation. These are not add-ons. They are core features people rely on daily. The patterns you are learning in this module, service layer architecture, error handling, caching, and prompt engineering, are the same patterns these companies use to ship AI reliably at scale.

Security and Privacy Considerations

As AI integration spreads, you must think carefully about what data you send to an external service and how you handle the response. A good habit is to sanitize input before it leaves your application.

```

/**
 * Sanitize user input before sending it to an AI service.
 */
private function sanitizeInputForAI(string $input): string
{
    // Redact patterns that look like sensitive information.
    $input = preg_replace('/\b\d{4}\s?\d{4}\s?\d{4}\s?\d{4}\b/', '[CARD_REDACTED]', $input);
    $input = preg_replace('/\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b/', '[EMAIL_REDACTED]', $input);
    $input = preg_replace('/\b\d{3}-\d{2}-\d{4}\b/', '[SSN_REDACTED]', $input);

    // Limit input length to prevent abuse.
    return substr(trim($input), 0, 1000);
}

```

⚠ Watch Out

Anything you send to an external AI provider leaves your servers. Never forward secrets, credentials, or personal data you would not want logged by a third party. Redaction and length limits like the ones above are a baseline, not a complete solution. Know your provider's data handling policy before you send real user data.

Performance and Graceful Degradation

An AI call is usually the slowest part of any AI-powered feature. Two strategies keep the experience good: move non-critical work to a background queue, and fall back to a useful default when the AI service is unavailable.

```

/**
 * Move non-critical AI work to a background queue.
 */
public function generateDraftAsync(string $title, int $userId): void
{
    GenerateAIDraftJob::dispatch($title, $userId);
}

/**
 * Fall back to a template when the AI service is unavailable.
 */

```

```

*/
public function generateBlogPostDraftWithFallback(string $title): string
{
    try {
        return $this->generateBlogPostDraft($title);
    } catch (\Throwable $e) {
        Log::warning('AI service unavailable, providing template fallback', [
            'title' => $title,
            'error' => $e->getMessage(),
        ]);

        return $this->generateTemplateContent($title);
    }
}

private function generateTemplateContent(string $title): string
{
    return "# {$title}

## Introduction
[Write an engaging introduction that hooks your readers and introduces the topic.]

## Main Content
[Develop your main points here, ideally in 2-3 subsections with clear headings.]

## Conclusion
[Summarize your key points and give readers a clear takeaway.];"
}

```

Testing AI Integrations

Testing code that calls an AI is tricky because responses vary. The solution is to fake the HTTP layer so your test controls exactly what the API returns. This lets you assert how your service behaves on success and on failure without spending a cent or waiting on the network.

```

// tests/Feature/AIIntegrationTest.php
<?php

namespace Tests\Feature;

```

```

use App\Models\User;
use App\Services\AiContentService;
use Illuminate\Support\Facades\Http;
use Tests\TestCase;

class AIIntegrationTest extends TestCase
{
    public function test_ai_service_handles_successful_response()
    {
        Http::fake([
            'api.openai.com/*' => Http::response([
                'choices' => [
                    ['message' => ['content' => 'This is a test blog post generated
                ],
            ], 200),
        ]);

        $service = new AiContentService();
        $result = $service->generateBlogPostDraft('Test Title');

        $this->assertStringContainsString('test blog post', $result);
    }

    public function test_ai_service_handles_api_failure()
    {
        Http::fake([
            'api.openai.com/*' => Http::response([], 500),
        ]);

        $service = new AiContentService();

        $this->expectException(\Exception::class);
        $service->generateBlogPostDraft('Test Title');
    }

    public function test_controller_handles_ai_generation_request()
    {
        $user = User::factory()->create();

        Http::fake([
            'api.openai.com/*' => Http::response([
                'choices' => [
                    ['message' => ['content' => 'Generated content']],
                ],
            ], 200),
        ]);

        $controller = new AiContentController();
        $response = $controller->generateBlogPostDraft($user, 'Test Title');

        $this->assertStringContainsString('Generated content', $response->getContent());
    }
}

```

```

        ],
    },
]);

$response = $this->actingAs($user)
    ->post('/posts/generate-draft', ['title' => 'Test Blog Post']);

$response->assertStatus(200);
$response->assertViewHas('generatedContent', 'Generated content');
}
}

```

⚙️ Professional Practice

`Http::fake()` is the reason you can test an AI feature at all. Because your service goes through Laravel's `Http` client, the test replaces the real network call with a response you define. This is a direct payoff of the service pattern: the boundary you built for clean architecture is the same boundary you mock for testing.

05 Building for Scale: Enterprise Patterns

AI Service Abstraction

As an application grows you may want to support more than one AI provider, or switch between them based on health or cost. An interface lets you code against a contract instead of a specific provider.

```

// app/Contracts/AIServiceInterface.php
<?php

namespace App\Contracts;

interface AIServiceInterface
{
    public function generateContent(string $input, array $options = []): string;
}

```

```

    public function isHealthy(): bool;
    public function getCostEstimate(string $input): float;
}

```

Our AI content service implements that contract. Note the cost estimate: it counts input tokens only and uses the current low-cost model's input rate. Real billing also charges for output tokens, which cost more, so treat this as a rough guide rather than an invoice.

```

// app/Services/AiContentService.php
class AiContentService implements AIServiceInterface
{
    // ... existing methods ...

    public function generateContent(string $input, array $options = []): string
    {
        $type = $options['type'] ?? 'blog_post';
        return $this->generateContent($input, $type);
    }

    public function isHealthy(): bool
    {
        try {
            $response = Http::timeout(5)->get($this->baseUrl . '/models');
            return $response->successful();
        } catch (\Throwable $e) {
            return false;
        }
    }

    public function getCostEstimate(string $input): float
    {
        // Rough estimate of INPUT cost only. Output tokens are billed
        // separately and cost more. Rates change often, so confirm against
        // OpenAI's current pricing page.
        $tokenCount = str_word_count($input) * 1.3; // Approximate tokens
        return $tokenCount * 0.00000015; // gpt-4o-mini input: $0.15 per 1M tokens
    }
}

```

A manager class can then pick a provider, check its health, and fail over to another if needed. Your application code calls the manager and never has to care which provider is behind it.

```
// app/Services/AIServiceManager.php
class AIServiceManager
{
    private array $services;
    private string $defaultService;

    public function __construct()
    {
        $this->services = [
            'openai' => new AiContentService(),
            // 'anthropic' => new AnthropicService(),
        ];

        $this->defaultService = config('ai.default_service', 'openai');
    }

    public function generateContent(string $input, array $options = []): string
    {
        $serviceName = $options['service'] ?? $this->defaultService;
        $service      = $this->getService($serviceName);

        if (!$service->isHealthy()) {
            $service = $this->getHealthyService(); // fall back to any healthy provider
        }

        return $service->generateContent($input, $options);
    }

    private function getService(string $name): AIServiceInterface
    {
        if (!isset($this->services[$name])) {
            throw new \InvalidArgumentException("AI service '{$name}' not found");
        }
        return $this->services[$name];
    }

    private function getHealthyService(): AIServiceInterface
    {
        // Implementation for finding a healthy service
    }
}
```

```

        foreach ($this->services as $service) {
            if ($service->isHealthy()) {
                return $service;
            }
        }
        throw new \Exception('No healthy AI services available');
    }
}

```

⚡ Modern PHP

This provider-swapping abstraction is exactly what `laravel/ai` and Prism give you out of the box, including automatic failover between providers. Building it once by hand, as here, is the best way to understand what those packages are doing for you when you adopt them.

Usage Analytics and Monitoring

In production you need to know how much AI you are using, what it costs, and how it performs. A small logging service writes one record per interaction so you can report on it later.

```

// app/Services/AIAnalyticsService.php
<?php

namespace App\Services;

use App\Models\AIUsageLog;

class AIAnalyticsService
{
    public function logUsage(string $service, string $type, string $input, string $output)
    {
        AIUsageLog::create([
            'user_id'      => auth()->id(),
            'service'      => $service,
            'type'         => $type,
            'input_length' => strlen($input),
            'output_length' => strlen($output),
        ]);
    }
}

```



```

        'cost'          ⇒ $cost,
        'created_at'    ⇒ now(),
    ]);
}

public function getUserUsageThisMonth(int $userId): array
{
    return AIUsageLog::where('user_id', $userId)
        →where('created_at', '≥', now()→startOfMonth())
        →selectRaw('COUNT(*) as requests, SUM(cost) as total_cost')
        →first()
        →toArray();
}
}

```

Resilience: The Circuit Breaker

If an external service starts failing, hammering it with more requests makes things worse. A circuit breaker stops calling a failing service for a cool-down period, then tries again. It is a professional resilience pattern you will meet in many systems, not just AI.

```

// A simplified circuit breaker for AI service reliability.
class AICircuitBreaker
{
    private int $failureThreshold;
    private int $timeout;
    private string $cacheKey;

    public function __construct(int $failureThreshold = 5, int $timeout = 300)
    {
        $this→failureThreshold = $failureThreshold;
        $this→timeout          = $timeout;
        $this→cacheKey          = 'ai_circuit_breaker';
    }

    public function call(callable $callback)
    {
        $state = cache()→get($this→cacheKey, ['failures' ⇒ 0, 'last_failure' ⇒ r

```

```

        // If the circuit is open, refuse until the timeout passes.
        if ($state['failures'] ≥ $this→failureThreshold) {
            if ($state['last_failure'] && now()→diffInSeconds($state['last_failure']
                throw new \Exception('AI service circuit breaker is open');
            }
            $state['failures'] = 0; // timeout passed, try to reset
        }

        try {
            $result = $callback();
            cache()→put($this→cacheKey, ['failures' ⇒ 0, 'last_failure' ⇒ null],
                return $result;
        } catch (\Throwable $e) {
            $state['failures']++;
            $state['last_failure'] = now();
            cache()→put($this→cacheKey, $state, 3600);
            throw $e;
        }
    }
}

```

06 Real-World Applications and Use Cases

The patterns in this module are not specific to a blog. The same service layer, prompt engineering, and error handling appear across industries. Here is where AI integration shows up most, and what it is used for.

Domain	Common AI-powered features
Content management	Draft generation, SEO and meta tags, summaries and excerpts, translation, tagging
E-commerce	Product descriptions, review summaries, recommendations, support chatbots

Domain	Common AI-powered features
Education	Feedback on writing, personalized paths, content adapted to skill level, quiz generation
Enterprise tools	Document summaries, meeting notes and action items, report drafting, workflow automation

Your blog feature demonstrates the core of all of these. The separation between business logic and AI service communication is what makes each of them reliable at scale, and it is the part employers care about most.

07

Assessment and Next Steps

What You Have Built

By finishing this module you have built an AI-powered feature that shows real professional competence: not just an API call, but architectural thinking, security awareness, performance sense, and testability. The patterns you used, service layer architecture, prompt engineering, error handling, caching, and monitoring, are the same ones production teams rely on. You now understand how to integrate an external AI service while keeping your code clean, maintainable, and easy to change.

The AI-Powered Development Landscape

The industry is in the middle of a significant shift, and developers who can combine solid web fundamentals with thoughtful AI integration are in a strong position. Your combination of PHP and Laravel fundamentals with modern AI integration is exactly the mix employers look for when they need to balance proven technology with new capability.

Best of all, the architecture transfers. The service layer you built for OpenAI works the same way for any future provider. The prompt engineering skills carry to new models. And the systematic approach to integration will serve you no matter which specific tools become dominant next. Demand for this skill reaches well beyond tech companies, into healthcare, finance, education, and retail.

Continuing Your Journey

The foundation you built opens several paths. You might explore image, voice, or domain-specific models. You might go deeper on the operations side: deployment, scaling, and monitoring of AI features. Or you might focus on the user experience of AI, making these features intuitive and genuinely useful. Whichever path you take, keep building complete, working applications that solve real problems. Each project should show growth in both depth and breadth. You now have the foundation and the confidence to help build the next generation of intelligent applications.

Key Takeaways

Prompt engineering is a transferable skill

Directing an AI well is now an expected competency inside broader roles, not a separate job title. Role prompting and clear, specific instructions are the core techniques.

The service layer is the whole point

Isolating all AI logic in one dedicated class keeps controllers thin, makes the code testable, and lets you swap providers without a rewrite.

Never hardcode secrets or stale models

API keys live only in `.env`. The model name lives there too (`gpt-4o-mini`), so you can move off a retiring model in one place.

Handle failure on purpose

Catch Throwable, log context, and degrade gracefully with fallbacks, caching, and a circuit breaker. An API failure should never break the user experience.

The framework now productizes this

Laravel 13 ships laravel/ai, and Prism does the same. Building the service by hand teaches the concepts; rebuilding on the SDK is the recommended next step.



Skills Self-Check

You are ready to move on to the capstone build if you can do each of these without looking back at the reading:

- ✓ Explain why an API key belongs in .env and never in your code or Git history.
- ✓ Describe the job of the service layer and why the controller should stay thin.
- ✓ Write a role prompt and add two specific instructions about length and format.
- ✓ Swap the AI model your application uses by changing a single line, and say which line.
- ✓ Explain the difference between catching Exception and catching Throwable.
- ✓ Fake the OpenAI HTTP call in a test and assert on both a success and a failure.
- ✓ Name one reason to cache an AI response and one reason to move the call to a queue.
- ✓ State what laravel/ai gives you that the hand-built service does not, and when to reach for it.

